TUTORIAL DE GIT

Capítulo 8. Personalización de Git

Juan Felipe Castañeda Rodríguez · Fundación Universitaria Agraria de Colombia (UNIAGRARIA)
Ciencia de Datos — Actividad 2: Hands on, configuración de herramientas · Octubre de 2026

Git se puede adaptar a las necesidades de cada persona o proyecto mediante configuración, atributos y scripts que se ejecutan ante ciertos eventos (hooks).

8.1 Configuración de Git

La configuración se lee de tres archivos, del más general al más específico (el más específico gana):

- /etc/gitconfig → nivel sistema (--system)
- ~/.gitconfig → nivel usuario (--global)
- .git/config → nivel repositorio (--local)

Opciones básicas del cliente:

```
git config --global core.editor "code --wait"
git config --global commit.template ~/.plantilla_commit.txt
git config --global core.pager "less -FRSX"
git config --global help.autocorrect 20
```

commit.template define un texto que aparece al escribir cada commit, útil para estandarizar mensajes.

Colores:

```
git config --global color.ui auto
git config --global color.diff.meta "blue black bold"
```

Herramientas externas de fusión y diferencias:

```
git config --global merge.tool meld
git config --global diff.tool vscode
```

Formato y espacios en blanco. Windows usa CRLF como fin de línea y Linux/macOS usan LF. Para evitar problemas:

```
git config --global core.autocrlf true
git config --global core.autocrlf input
```

La primera opción se usa en Windows; la segunda en macOS/Linux. core.whitespace permite detectar espacios sobrantes al final de línea.

Configuración del servidor: opciones como receive.fsckObjects (verificar objetos recibidos), receive.denyNonFastForwards (impedir push forzados) y receive.denyDeletes (impedir borrar ramas).

8.2 Atributos de Git

En un archivo .gitattributes se definen comportamientos por ruta.

Archivos binarios:

```
*.pdf binary
*.docx diff=word
```

Con un controlador de diff configurado, Git puede mostrar diferencias legibles incluso en documentos de Word, convirtiéndolos a texto:

```
git config diff.word.textconv docx2txt
```

Expansión de palabras clave y filtros. Los filtros clean y smudge transforman archivos al guardarlos y al extraerlos (por ejemplo, formatear código automáticamente).

Exportar el repositorio: con export-ignore se excluyen archivos al generar un archivo con git archive.

Estrategias de fusión: un atributo como merge=ours hace que un archivo concreto siempre conserve la versión propia al fusionar.

8.3 Hooks (puntos de enganche)

Los hooks son scripts que Git ejecuta automáticamente antes o después de ciertos eventos. Se guardan en .git/hooks; hay ejemplos con la extensión .sample. Para activar uno, se le quita la extensión y se le dan permisos de ejecución.

Hooks del lado del cliente:

- **pre-commit:** se ejecuta antes de crear el commit; sirve para correr pruebas o un linter y cancelar el commit si fallan.
- **prepare-commit-msg** y **commit-msg:** permiten editar o validar el mensaje del commit (por ejemplo, exigir un formato).
- **post-commit:** notificaciones después de confirmar.
- **pre-push:** validaciones antes de enviar al remoto.
- **pre-rebase, post-checkout, post-merge:** reaccionan a esas operaciones.

Hooks del lado del servidor:

- **pre-receive:** puede rechazar todo un push.
- **update:** se ejecuta una vez por cada rama actualizada.
- **post-receive:** se usa para notificaciones o despliegues automáticos.

Los hooks no se copian al clonar el repositorio, por lo que se suelen distribuir mediante scripts o herramientas aparte.

8.4 Ejemplo de política implementada por Git

Combinando hooks se pueden aplicar reglas como: todo mensaje de commit debe incluir un número de incidencia, ciertos usuarios solo pueden modificar ciertas carpetas, o no se permiten push que no sean fast-forward. Por ejemplo, un hook commit-msg puede verificar con una expresión regular que el mensaje contenga algo como "ref: #123" y rechazar el commit si no lo tiene.

8.5 Resumen

Con git config, .gitattributes y los hooks se puede ajustar Git a cada flujo de trabajo: editor, colores, finales de línea, manejo de binarios, validaciones automáticas y políticas del equipo.

Referencia

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2